📑 Basic Information:

Experiment No: 14

Date:

Title: Checking Balanced Parentheses Using a Stack in Java.

🎯 Objective /Aim:

To write a Java program that uses a stack to determine whether the parentheses and brackets in an input string are correctly balanced and matched.

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string

is valid. An input string is valid if:

1. Open brackets must be closed by the same type of brackets.

2. Open brackets must be closed in the correct order.

Note that an empty string is also considered valid.

Input: ( )

Input: ({) }

🛠️ System Requirements:

Hardware:

-> Standard PC/Laptop

-> Intel Core i3/i5 processor or equivalent

-> Minimum 4 GB RAM

-> Minimum 500 MB free

Software:

->Operating System: Windows / Linux / macOS

-> Java Development Kit (JDK 17 or JDK 21)

-> Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code

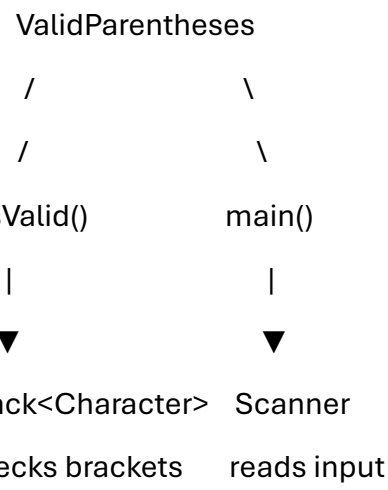
-> Command Prompt / Terminal

Algorithm/Logic Description:

1. Start the program.

2. Read a string from the user.
3. Create an empty stack.
4. Scan each character in the string:
    - If it is an opening bracket (, {, or [, push it onto the stack.
    - If it is a closing bracket:
        - If the stack is empty, mark the string invalid.
        - Otherwise, pop the top opening bracket and check whether it matches the closing bracket.
        - If they do not match, mark the string invalid.
5. After scanning the string, check whether the stack is empty.
    - If it is empty and all brackets matched, display "Valid."
    - Otherwise, display "Not valid."
6. Close the scanner and stop the program.

UML Class Diagram:



Source Code:

```
import java.util.Stack;
import java.util.Scanner;
// Checks whether brackets in a string are balanced
public class ValidParentheses {
  // Returns true if every opening bracket has a matching closing bracket
  public static boolean isValid(String s) {
```

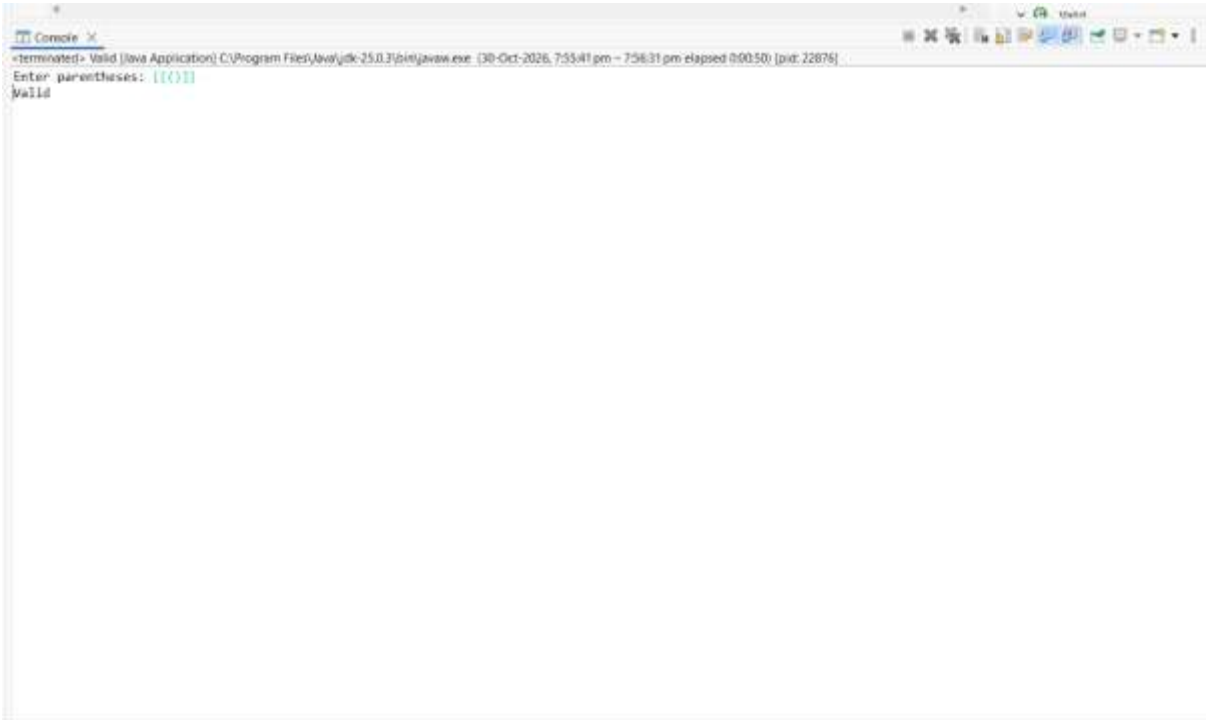
```
// Stores opening brackets until their matching closing brackets appear
Stack<Character> stack = new Stack<>();
// Examine each character in the input string
for (int i = 0; i < s.length(); i++) {
  char ch = s.charAt(i);
  // Push opening brackets onto the stack
  if (ch == '(' || ch == '{' || ch == '[') {
    stack.push(ch);
  }
  // Process a closing bracket
  else {
    // A closing bracket cannot match if there is no opening bracket
    if (stack.isEmpty()) {
      return false;
    }
    // Get the most recent opening bracket
    char top = stack.pop();
    // Check that the closing parenthesis matches
    if (ch == ')' && top != '(') {
      return false;
    }
    // Check that the closing brace matches
    if (ch == '}' && top != '{') {
      return false;
    }
    // Check that the closing square bracket matches
    if (ch == ']' && top != '[') {
      return false;
    }
```

```
            }
        }
        // The string is valid only if no unmatched opening brackets remain
        return stack.isEmpty();
    }
    public static void main(String[] args) {
        // Read the bracket string from the user
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter parentheses: ");
        String s = sc.nextLine();
        // Check the string and display the result
        if (isValid(s)) {
            System.out.println("Valid");
        } else {
            System.out.println("Not valid");
        }
        // Close the scanner
        sc.close();
    }
}
```

Relationship:

ValidParentheses uses a Stack<Character> in isValid() to check whether brackets are balanced. The main() method uses Scanner to read the input, calls isValid(), and displays whether the brackets are valid.

Output Screenshot:



Result:

The program uses a stack to check whether each opening bracket has a matching closing bracket in the correct order. It reports the input as valid when all brackets match and no unmatched brackets remain.